



Complete Summary: Time & Space Complexity

This document is a comprehensive summary of the provided video lecture on Time and Space Complexity, designed for Data Structures and Algorithms (DSA) and placement preparation.

1. Introduction: Why Study Time & Space Complexity?

Whenever you write code, interviewers will ask for its Time and Space Complexity. It is the standard way to measure how efficient an algorithm is. We don't measure efficiency in actual seconds because a program will run faster on a supercomputer than on an old laptop. Instead, we measure how the code behaves as the **input size grows**.

2. Time Complexity

Definition: Time complexity is the amount of time taken by an algorithm as a function of the input size (n). We measure this in terms of the **number of operations** performed, not seconds.

- **Big O Notation (O):** This is the mathematical symbol used to represent the **worst-case scenario** (upper bound) of an algorithm. We care most about the worst case because we want to know how our code performs when things are at their absolute worst (e.g., finding an element that is at the very end of an array).
- **How to Calculate Big O:**
 1. **Ignore Constants:** If an algorithm takes $O(4n^2 + 3n + 5)$ steps, ignore the numbers (4, 3, 5).
 2. **Take the Largest Term:** As n gets massive (like 1 million), n^2 becomes infinitely larger than n . So, drop the smaller terms. The final complexity becomes simply $O(n^2)$.

3. Space Complexity

Definition: It is the amount of **extra space (memory)** an algorithm takes as a function of the input size.

- **Input Space vs. Auxiliary Space:** If you are given an array of size n , that is the *Input Space*. You don't count that. You only count the *Auxiliary Space*, which is any **new** variables, arrays, or data structures you create inside your code to solve the problem.
- **Example:** If you iterate through a list and keep a single `sum` variable, your extra space is $O(1)$ (Constant Space). If you create a brand new copy of the array, your extra space is $O(n)$ (Linear Space).

4. Common Time Complexities (From Best to Worst)

Here are the most common complexities you will encounter:

1. $O(1)$ - **Constant Time (Best):** The operations do not depend on the input size. Example: Doing a simple math calculation (like finding the sum of n numbers using the formula $n * (n+1) / 2$) or accessing the first element of an array.
2. $O(\log n)$ - **Logarithmic Time (Excellent):** The search space gets divided in half at every step. Example: Binary Search.
3. $O(n)$ - **Linear Time (Good):** The operations scale exactly with the input size. Example: A single `for` loop (like counting elements or Linear Search).

4. $O(n \log n)$ - **Linearithmic Time (Fair)**: Usually seen in efficient sorting algorithms. Example: Merge Sort, Quick Sort (average case).
5. $O(n^2)$ & $O(n^3)$ - **Quadratic & Cubic Time (Slow)**: Found in unoptimized algorithms with nested loops (a loop inside a loop). Example: Bubble Sort, Selection Sort, printing all sub-arrays.
6. $O(2^n)$ - **Exponential Time (Very Slow)**: Often seen in brute-force recursion where each step branches into multiple paths. Example: Calculating Fibonacci numbers using simple recursion.
7. $O(n!)$ - **Factorial Time (Terrible)**: The slowest. Example: Finding all possible permutations of a string.

5. Time & Space Complexity in Recursion

Calculating complexity for recursive functions is slightly different than regular loops.

- **Time Complexity: * Formula:** Total Number of Calls $\times$ Work Done in Each Call .
 - You figure this out by drawing a **Recursion Tree**.
 - *Example (Fibonacci)*: Each call makes 2 more calls. The tree depth is n . The total calls are roughly 2^n . If the work in each call is constant ($O(1)$), the total time is $O(2^n)$.
 - *Note*: You can also calculate this using Mathematical Recurrence Relations.
- **Space Complexity: * Formula:** Maximum Depth of Recursion Tree $\times$ Memory in Each Call .
 - Even if you don't create variables, recursion uses memory behind the scenes in the **Call Stack**.
 - *Example (Factorial)*: A factorial of n makes n nested calls. The stack height is n . Therefore, the space complexity is $O(n)$.

6. Practical Usage (The "Cheat Sheet")

When solving problems on platforms like LeetCode or in coding assessments, you are usually given a time limit (around 1 second, which equals roughly 10^8 operations). You can look at the **Constraints** of the problem to guess what Time Complexity your solution needs to be to avoid a TLE (Time Limit Exceeded) error!

- If $N \leq 10^8$: You must write an $O(n)$ or $O(\log n)$ solution.
- If $N \leq 10^5$: You can get away with an $O(n \log n)$ solution (Hint: You might need to Sort!).
- If $N \leq 10^4$: An $O(n^2)$ solution will work (Nested loops).
- If $N \leq 500$: An $O(n^3)$ solution will work (Three nested loops).
- If $N \leq 25$: An $O(2^n)$ solution will work (Hint: Use Recursion).
- If $N \leq 12$: An $O(n!)$ solution will work (Hint: Permutations/Backtracking).

Summary Conclusion

Time and Space complexity are the foundation of Data Structures and Algorithms. Calculating them is a skill that takes practice, and you will learn to apply these rules naturally as you write more code and analyze different algorithms!